

Procedural Music System for Unity

A fully procedural, real-time music generation system for Unity. No pre-recorded samples — all audio is generated from scratch using oscillators, filters, and Karplus-Strong synthesis. Designed for a Scandinavian folklore survival game inspired by Over the Garden Wall, Don't Starve, and Minecraft.

Built on **Lerdahl's Tonal Pitch Space** theory for harmonically coherent chord progressions that respond dynamically to game state and tension.

Quick Start

1. Create an empty GameObject in your scene
2. Add an **AudioSource** component (no clip needed)
3. Add the **ProceduralMusicController** component
4. Optionally add **FullSystemTest** for the ImGui debug panel
5. Call `SetTension()` and `SetGameState()` from your game code

```
csharp
var music = FindObjectOfType<ProceduralMusicController>();
music.SetGameState(GameMusicState.Exploring);
music.SetTension(0.3f);
```

Game States

Key	State	Mode	Tempo	Character
1	Exploring	Dorian	78-100	Folk wandering through Scandinavian woods

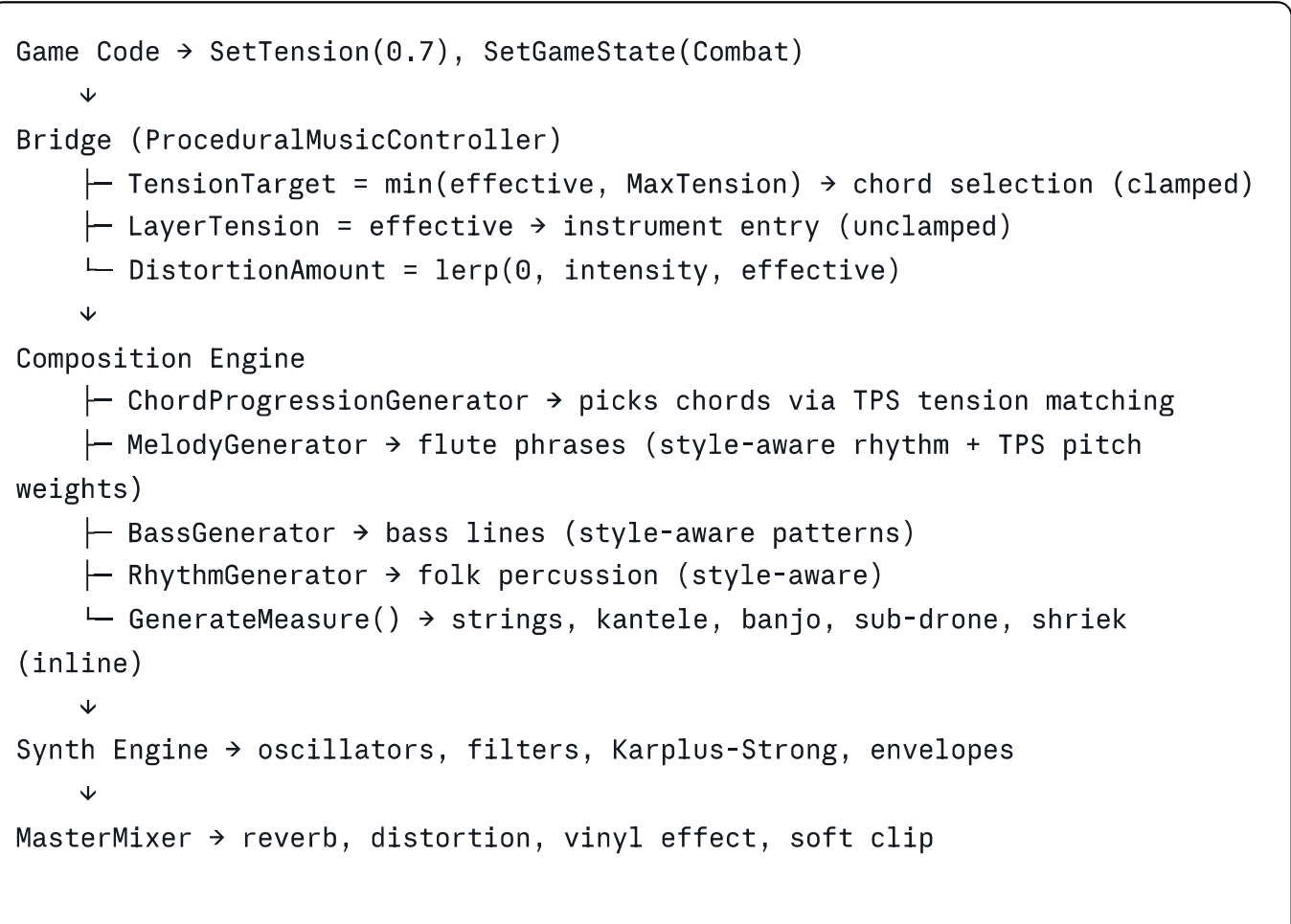
Key	State	Mode	Tempo	Character
2	Exploring2	Major	90-112	Found safety — brighter, confident
3	Pressure	Natural Minor	85-110	Something stalking — distortion builds
4	Combat	Harmonic Minor	85-110	Fighting creatures — heavy, atmospheric
5	Spooky	Phrygian	55-75	Nisse watching — tritones, unsettling
6	Horror	Locrian	40-20	The beast — broken vinyl, ambient dread
7	Night	Aeolian	55-85	Campfire to terror arc with tension
8	Cozy	Major	62-78	By the fire — banjo fingerpicking, warm

Instruments

Index	Name	Synth Type	Description
0	HurdyGurdy	HurdyGurdy	Square wave drone + wheel buzz + pitch wobble
1	Flute	LegatoLead	Breathy additive sine + portamento + delayed vibrato
2	Bass	Subtractive	Saw + sub-sine octave below, dark filter
3	Log Drum	Percussion	Deep woody thump (kick replacement)
4	Frame Drum	Percussion	Skin slap — bodhran style (snare replacement)
5	Brush	Percussion	Stick on wood texture (hi-hat replacement)
6	Folk Strings	StringEnsemble	4 detuned saws, dark filter, slow folk vibrato

Index	Name	Synth Type	Description
7	Kantele	PluckedString	Karplus-Strong Finnish zither — folk arpeggios
8	Sub Drone	Subtractive	Ultra-low rumble, Horror only
9	Shriek String	StringEnsemble	High-pitched horror string, semitone/tritone crawl
10	Banjo	PluckedString	Bright twangy fingerpicking, Cozy only

How It Works





Unity AudioSource (OnAudioFilterRead)

Layer System

Each instrument has a **LayerRange** per state that controls when it plays based on tension:

```
csharp
LayerRange.Always          // Always on
LayerRange.Off             // Never plays
LayerRange.From(0.4f)      // Enters at tension 0.4
LayerRange.Until(0.5f)     // Plays at start, drops out at 0.5
LayerRange.Between(0.2f, 0.7f) // Only active in that range
```

This means instruments layer in (or out) gradually as tension changes, and each state can have completely different layering behavior.

Music Style System

Each state sets a **MusicStyle** that controls rhythm patterns for ALL instruments:

Style	Character	States
Folk	Dotted rhythms, lilting triplets, flowing arpeggios	Exploring, Cozy
Tense	Syncopated, driving eighth notes, tremolo	Pressure, Combat
Sparse	Long held notes, lots of silence, ambient	Spooky, Night
Triumphant	Bold march-like quarters, strong downbeats	Exploring2
Horror	Almost nothing — rare broken phrases, dissonant	Horror

Styles can be set per-instrument if needed (e.g. Folk kantele with Tense strings).

Effects

- **Distortion** — atan waveshaper + bitcrushing. Scales with tension. Enabled per state.
- **Vinyl** — Wow/flutter + crackle + LP darkening. Makes audio sound like a broken record. Horror only.
- **Reverb** — Comb filter (50ms delay, 0.4 feedback, 0.15 wet mix).

Public API

csharp

```

// Core control
SetTension(float 0-1) // Continuous tension parameter
SetGameState(GameMusicState) // Switch music state
SetKey(PitchClass) // Change key root, keep mode
ForceModulation(PitchClass, MusicalMode) // Change key root + mode
SetDistortion(bool) // Toggle distortion

// Layer control
SetLayerEnabled(bool pad, bool melody, // Simple on/off
                bool bass, bool perc, bool strings,
                bool kantele)
SetLayerRanges(LayerRange pad, ...) // Advanced per-layer ranges

// Info
GetCurrentMusicalTension() // Read TPS tension of current chor
GetCurrentChord() // Read current chord
GetInstruments() // Access voice managers directly
GetComposer() // Access composition engine

// Emergency
Panic() // Kill all sound immediately

```

Test Controls

With `MusicTestController` attached:

Key	Action
1-8	Switch game states
↑/↓	Increase/decrease tension
Q/W	Change key root down/up
M/N	Switch to Major/Minor
Space	Panic (silence all)
D	Toggle debug display

With `FullSystemTest` attached, an ImGui panel provides: controls, synth test keyboard, TPS visualizer, scenario runner, waveform display, CPU performance monitor, and voice counts.

File Structure

```
Scripts/  
  Core/  
    TonalPitchSpace.cs      Scales, chords, tension, attraction, chord  
  suggestions  
    Composition/  
      CompositionEngine.cs  Melody, bass, rhythm, strings, kantele,  
  banjo, horror generators  
    Synthesis/  
      SynthEngine.cs        Oscillators, envelopes, filters, FM, Karplus-  
  Strong, hurdy-gurdy  
      VoiceManager.cs       Instrument presets, voice pooling, master  
  mixer, effects
```

Bridge/

ProceduralMusicController.cs MonoBehaviour, state configs, game API

Examples/

FullSystemTest.cs IMGUI debug/test panel

MusicTestController.cs Keyboard shortcut controller